

Informe final — Predicción de cancelaciones hoteleras

Máster en Inteligencia Artificial, Cloud Computing y DevOps — PontIA
Módulo de Machine Learning y Deep Learning

Autores: Jorge Pozo Galán · Peio Murguía · Pablo Calderón
Repositorio: https://github.com/PMurguia/proyecto_ml_pontia
Fecha defensa: 01/10/2026

1. Roles del equipo

Integrante	Responsabilidades	Aportación principal
Jorge Pozo Galán	Regresión logística y árbol de decisión. Diseño del preprocesado con one-hot encoding y escalado. Justificación de F1-Score como métrica principal. Visualización con tabla comparativa integrada.	Definición de la métrica principal del equipo y estructura de evaluación común (matriz de confusión + curva ROC por modelo).
Peio Murguía	Red neuronal multicapa con Keras/TensorFlow. Arquitectura de capas, regularización mediante dropout, early stopping sobre <code>val_loss</code> . Exportación de métricas a CSV.	Implementación del único modelo de deep learning del proyecto y automatización de la salida de resultados.
Pablo Calderón	Random Forest y XGBoost. Análisis exploratorio y detección de la fuga de información. Pipeline con <code>ColumnTransformer</code> y codificación ordinal. Comparativa con AutoML (PyCaret).	Identificación de <code>reservation_status</code> y <code>reservation_status_date</code> como fuga, decisión que condicionó todo el proyecto.

2. Justificación del problema

2.1 Contexto de negocio

En el sector hotelero, entre el 30 % y el 40 % de las reservas se cancelan. Cada cancelación no anticipada supone una habitación que queda vacía sin haberse revendido. Predecirlas permite:

- **Gestionar el overbooking** con fundamento estadístico en lugar de por intuición.
- **Aplicar precios dinámicos:** elevar el precio o exigir depósito en segmentos con alta probabilidad de cancelación.
- **Planificar personal y compras** con una previsión de ocupación más fiable.

2.2 Por qué es un problema de clasificación binaria

La variable objetivo `is_canceled` toma exactamente dos valores (0 y 1), y la decisión de negocio es binaria: se actúa sobre la reserva o no se actúa. No hay categorías intermedias ni una magnitud continua que predecir.

2.3 Por qué este conjunto de datos

- **Volumen suficiente:** 119.390 registros permiten entrenar modelos complejos (redes neuronales, conjuntos de cientos de árboles) sin que el sobreajuste sea inevitable.
- **Variedad de tipos:** mezcla variables numéricas continuas (`adr`, `lead_time`), enteras (`adults`, `booking_changes`), categóricas de baja cardinalidad (`hotel`, `meal`) y de alta cardinalidad (`country` con 177 valores, `agent` con 333). Obliga a tomar decisiones reales de preprocesado.
- **Problemas realistas:** contiene nulos, duplicados, outliers y, sobre todo, **fuga de información**, que es el tipo de dificultad que aparece en un proyecto profesional y no en un dataset de laboratorio.
- **Desbalance moderado** (63/37): suficiente para que la accuracy resulte engañosa y haya que razonar sobre la métrica adecuada.

3. Análisis exploratorio de datos

3.1 Descripción general

Aspecto	Valor
Filas	119.390
Columnas	32 (31 predictoras + 1 objetivo)
Balance del target	62,96 % no canceladas / 37,04 % canceladas
Duplicados exactos	31.994 (26,8 %)
Rango temporal	julio 2015 - agosto 2017
Tipos	12 categóricas (texto), 20 numéricas

3.2 Valores ausentes

Columna	Nulos	%	Tratamiento	Justificación
company	112.593	94,3 %	Imputación por moda / binaria has_company	Con un 94 % de ausencias, como identificador aporta ruido más que señal
agent	16.340	13,7 %	Categoría propia	La ausencia de intermediario es informativa: indica reserva directa
country	488	0,4 %	Categoría "Desconocido"	El hecho de faltar puede correlacionar con el canal de reserva
children	4	<0,01 %	0	Es la moda y tiene sentido de negocio

Decisión de equipo: ninguna fila se elimina por nulos. Todos los pipelines incorporan un SimpleImputer , de modo que la imputación se ajusta solo con los datos de entrenamiento.

3.3 Detección de la fuga de información

Resultado principal — tabla cruzada de reservation_status :

reservation_status	is_canceled = 0	is_canceled = 1
Check-Out	75.166	0
Canceled	0	43.017
No-Show	0	1.207

Correspondencia perfecta: la variable es el target reetiquetado. Incluirla produce un AUC de 1,0 y un modelo sin ningún valor predictivo. reservation_status_date filtra por la misma vía: para una cancelación es la fecha en que se canceló, anterior a la llegada; para una estancia completada es la fecha de salida. La diferencia con la fecha de llegada reconstruye el target.

Hallazgos secundarios (fuga temporal parcial):

Variable	Observación	Interpretación
required_car_parking_spaces	De 7.416 reservas que piden parking, 0 cancelan	Separación perfecta. Sugiere que la plaza se registra en recepción, es decir, al llegar
assigned_room_type	La habitación se asigna en el check-in	Momento posterior al de predicción
booking_changes	Acumula modificaciones hasta el desenlace	Parte de la información es posterior a la reserva

Estas tres se conservaron para no perder capacidad predictiva, pero se documentan como limitación.

3.4 Distribución del target por variables categóricas

Las tasas de cancelación por categoría orientaron la selección de variables:

deposit_type — la variable más discriminante:

Categoría	Tasa de cancelación	Nº reservas
Non Refund	99,4 %	14.587
No Deposit	28,4 %	104.641
Refundable	22,2 %	162

Un depósito no reembolsable cancela casi siempre. No es fuga —se conoce al reservar— pero domina cualquier modelo, lo que se refleja en su alta importancia.

market_segment :

Categoría	Tasa de cancelación	Nº reservas
Groups	61,1 %	19.811
Online TA	36,7 %	56.477
Offline TA/TO	34,3 %	24.219
Corporate	18,7 %	5.295
Direct	15,3 %	12.606
Complementary	13,1 %	743

Los grupos cancelan cuatro veces más que las reservas directas.

customer_type : Transient 40,7 % frente a Group 10,2 %. **hotel** : City Hotel 41,7 % frente a Resort Hotel 27,8 %. **is_repeated_guest** : clientes nuevos 37,8 % frente a repetidores 14,5 %.

3.5 Variables numéricas

lead_time (días entre reserva y llegada) — la más informativa de las numéricas:

	Media	Mediana	Percentil 75
No canceladas	80,0	45	124
Canceladas	144,8	113	214

Las reservas que se cancelan se hacen con casi el doble de antelación. Cuanto antes se reserva, más tiempo hay para cambiar de planes.

total_of_special_requests — relación monótona inversa:

Peticiones especiales	Tasa de cancelación
0	47,7 %
1	22,0 %
2	22,1 %
3	17,9 %
4 o más	≤ 10,6 %

Interpretación: pedir algo concreto revela compromiso con el viaje.

3.6 Valores atípicos y registros inválidos

Variable	Anomalía	Decisión
adr	1 valor negativo (−6,38)	Candidato a eliminación
adr	1 valor de 5.400 (media 101,8)	Error de tecleo evidente
Huéspedes	180 reservas con 0 personas	Registros sin sentido de negocio
Noches	715 reservas con 0 noches	Conservadas: son day-use o no-shows legítimos
babies	Máximos de 9 y 10	Conservados: no afectan a modelos de árboles

3.7 Estabilidad temporal

Año	Tasa de cancelación
2015	37,0 %
2016	35,9 %
2017	38,7 %

La tasa es estable, lo que descarta una deriva brusca del fenómeno. No obstante, la validación temporal (sección 7) muestra que el rendimiento sí cae al predecir un año no visto.

3.8 Decisiones derivadas del EDA

Hallazgo del EDA	Decisión tomada
<code>reservation_status</code> reproduce el target	Eliminada en los tres scripts
<code>reservation_status_date</code> delata el desenlace	Eliminada en los tres scripts
<code>country</code> (177) y <code>agent</code> (333) de alta cardinalidad	Ordinal encoding en modelos de árboles; one-hot en lineales y red
94 % de nulos en <code>company</code>	Imputación por moda; no aporta como identificador
<code>adr</code> con outliers extremos	Imputación por mediana, robusta frente a ellos
Desbalance 63/37	F1-Score como métrica principal en lugar de accuracy
26,8 % de duplicados	Conservados; se documenta el efecto sobre el split aleatorio

4. Diseño del sistema

4.1 Arquitectura

```
CSV crudo (data/raw/)
├── Eliminar columnas con fuga
├── Separar X / y
├── train_test_split(test_size=0.2, stratify=y, random_state=42)
└── Pipeline([ColumnTransformer, modelo])
    ├── numéricas: SimpleImputer(mediana) + StandardScaler
    ├── categóricas: SimpleImputer(modas) + encoder
    └── métricas + matriz de confusión + curva ROC → outputs/
```

El sistema se organiza en tres scripts independientes, uno por integrante, con una estructura común acordada: mismo tratamiento de la fuga, misma partición y mismo conjunto de métricas. Esto permite comparar los resultados entre modelos sin que las diferencias provengan de la metodología.

4.2 Por qué un Pipeline

Todos los modelos se envuelven en un `Pipeline` de scikit-learn. La razón es evitar una fuga sutil: si se codifica o se imputa **antes** de dividir, el transformador ve los datos del conjunto de test y aprende de ellos. Dentro del pipeline, `fit` ajusta el preprocesado solo con el train y `predict` lo aplica al test sin reajustarlo.

Beneficio adicional: el objeto serializado incluye el preprocesado, de modo que para predecir sobre datos nuevos basta con pasarle un CSV crudo.

4.3 Decisiones técnicas

Decisión	Alternativa descartada	Motivo
<code>Pipeline</code> de scikit-learn	Preprocesar antes de dividir	Evita que el encoder vea el test
Ordinal encoding (árboles)	One-hot	<code>country</code> y <code>agent</code> generarían ~500 columnas dispersas; un árbol reagrupa categorías por su cuenta
One-hot (lineales y red)	Ordinal	Un modelo lineal interpretaría el código como magnitud
Imputación por mediana	Media	Robusta frente a los outliers de <code>adr</code>
<code>stratify=y</code>	Split aleatorio simple	Mantiene el 63/37 en ambos conjuntos
<code>random_state=42</code> en todo	Sin semilla	Reproducibilidad de los resultados
Conservar duplicados	<code>drop_duplicates()</code>	Se consideran reservas de grupo reales; se documenta el efecto

4.4 Modelos implementados

Modelo	Librería	Configuración
Regresión logística	scikit-learn	<code>max_iter=1000</code> , con <code>StandardScaler</code>
Árbol de decisión	scikit-learn	<code>random_state=42</code> , sin poda
Red neuronal	Keras / TensorFlow	Densa 64 → Dropout 0,3 → Densa 32 → Dropout 0,2 → Sigmoide. Adam, <code>binary_crossentropy</code> , 20 épocas, batch 256, early stopping con paciencia 5 sobre <code>val_loss</code>
Random Forest	scikit-learn	<code>n_estimators=300</code> , <code>min_samples_leaf=2</code> , <code>max_features="sqrt"</code>
XGBoost	xgboost	<code>n_estimators=300</code> , <code>learning_rate=0.1</code> , <code>max_depth=6</code>
Dummy (referencia)	scikit-learn	<code>strategy="most_frequent"</code>

5. Métrica principal y su justificación

Se adopta **F1-Score** como métrica principal común a los cinco modelos.

Por qué no accuracy: con un 62,96 % de reservas no canceladas, un clasificador que siempre responda "no cancela" alcanza ese 62,96 % sin haber aprendido nada. El `DummyClassifier` incluido en el proyecto lo demuestra empíricamente. Cualquier accuracy debe leerse contra ese suelo.

Por qué F1: combina precisión y recall sobre la clase minoritaria mediante una media armónica, que penaliza el desequilibrio entre ambas. Un modelo con precisión 0,99 y recall 0,10 obtendría 0,55 con una media aritmética, pero solo 0,18 con F1.

ROC-AUC como métrica secundaria: tiene una ventaja que F1 no tiene, la independencia del umbral. Las métricas basadas en la predicción binaria se calculan tras aplicar un corte de 0,5 sobre la probabilidad, y ese 0,5 es una convención. El AUC mide la capacidad de **ordenar** correctamente las reservas por riesgo, con independencia del corte.

Lectura de negocio de precisión y recall:

- **Falso negativo** (predecimos que viene y cancela): habitación vacía sin revender. Coste directo del ingreso perdido.
- **Falso positivo** (predecimos cancelación y viene): riesgo de overbooking. Coste de realojar al cliente y daño reputacional.

Ambos importan, pero en la mayoría de hoteles el primero es más frecuente y más fácil de cuantificar, lo que justifica prestar atención al recall.

6. Resultados

Partición 80/20 estratificada sobre 23.878 reservas de test, semilla 42.

Modelo	F1-Score	Accuracy	Precisión	Recall	ROC-AUC
Regresión logística	0,7302	0,8182	0,8112	0,6639	0,8961
Árbol de decisión	0,8111	0,8593	0,8066	0,8157	0,8528
Red neuronal (Keras)	0,8123	0,8697	0,8708	0,7612	0,9454
XGBoost	0,8368	0,8826	0,8625	0,8127	0,9543
Random Forest	0,8486	0,8929	0,8904	0,8105	0,9595
Dummy (referencia)	0,0000	0,6296	0,0000	0,0000	0,5000

6.1 Elección final

Se selecciona el **Random Forest** como modelo del sistema. Obtiene el F1-Score más alto (0,8486), la mejor precisión (0,8904) y el mejor ROC-AUC (0,9595), siendo además el segundo en recall.

Por qué supera a los demás en este caso concreto:

- **Frente a la regresión logística** (+11,8 puntos de F1): el modelo lineal no puede representar interacciones. La probabilidad de cancelación de una reserva con mucha antelación depende de si es un grupo, de si hay depósito no reembolsable y del país de origen, de forma conjunta y no aditiva. Se ve en el recall: 0,6639 frente a 0,8105, es decir, la regresión pierde una de cada tres cancelaciones.
- **Frente al árbol de decisión** (+3,8 puntos de F1): el promedio de 300 árboles decorrelacionados reduce la varianza. Un árbol único memoriza casos concretos del train; el bosque promedia esos errores y se cancelan entre sí.
- **Frente a la red neuronal** (+3,6 puntos de F1): con datos tabulares de tamaño medio, los conjuntos de árboles suelen igualar o superar a las redes densas, que necesitan más datos y más ajuste para destacar.
- **Frente a XGBoost** (+1,2 puntos de F1): la diferencia es la menor de todas y se obtiene con hiperparámetros por defecto. No constituye una superioridad estructural del algoritmo; con búsqueda de hiperparámetros el orden podría invertirse.

6.2 Matriz de confusión del modelo seleccionado

	Predicho: no cancela	Predicho: cancela
Real: no cancela	14.151 (VN)	882 (FP)
Real: cancela	1.676 (FN)	7.169 (VP)

Traducido a negocio: 1.676 habitaciones quedarían vacías sin previsión, frente a 882 casos de posible overbooking.

7. Reflexión crítica: limitaciones y mejoras

7.1 Limitaciones de los datos

Duplicados. 31.994 filas idénticas en las 32 columnas (26,8 %). Se conservaron por interpretarse como reservas de grupo reales, pero es una decisión discutible con impacto directo en las métricas.

Variables con fuga temporal parcial. `required_car_parking_spaces` separa perfectamente (0 cancelaciones de 7.416 reservas con parking), lo que sugiere registro en recepción. Se conservó, pero un análisis de robustez entrenando sin ella cuantificaría cuánto depende el modelo de ese artefacto.

Dominancia de `deposit_type`. La categoría "Non Refund" cancela el 99,4 % de las veces. Es información legítima y disponible al reservar, pero concentra tanta señal que tapa la contribución del resto de variables.

Alcance geográfico y temporal. Dos hoteles portugueses entre 2015 y 2017. La generalización a otros mercados o a un periodo posterior no está comprobada.

7.2 Mejoras propuestas

Mejora	Beneficio esperado
<code>StratifiedKFold(5)</code> en los tres scripts	Convierte la principal limitación en un punto fuerte y permite afirmar diferencias con solidez
<code>RandomizedSearchCV</code> para los cinco modelos	Comparación justa con hiperparámetros optimizados
Target encoding para <code>country</code> y <code>agent</code>	Aprovecha la alta cardinalidad sin explosión dimensional
Calibración de probabilidades (<code>CalibratedClassifierCV</code>)	Mejoraría especialmente al árbol de decisión, cuyo bajo AUC procede de probabilidades mal calibradas
Optimización del umbral por coste económico	Con ingresos por habitación y coste de overbooking, maximizar beneficio esperado en lugar de F1
Análisis de robustez sin variables sospechosas	Cuantifica la dependencia de posibles artefactos
SHAP para interpretabilidad local	Explicar por qué una reserva concreta se marca como riesgo
Reentrenamiento periódico	Contrarrestar la deriva temporal detectada
API REST con FastAPI	Exponer el modelo para integración con el sistema de reservas